



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных
технологий

Отчет по практической работе №2

по дисциплине

«Технология разработки программных приложений»

Тема: «Основы работы со скриптами»

Выполнил:

Студент группы ИНБО-20-23

Боргачев Тимофей Максимович

Проверил:

ассистент Петрова А.А.

Москва 2025

ОГЛАВЛЕНИЕ

ПРАКТИЧЕСКАЯ РАБОТА №2. ОСНОВЫ РАБОТЫ С BASH SCRIPT`АМИ	3
Введение.....	3
Часть 1. Базовые Bash скрипты.....	3
Переменные.....	4
Подстановка команд	4
Математические операции.....	5
Управляющая конструкция if-then-else	6
Сравнение чисел.....	7
Сравнение строк	8
Проверки файлов.....	9
Операторы цикла.....	10
Выполнение заданий	14
Часть 2. Развертка и запуск проекта при помощи Bash Script.....	19
Определение зависимостей проекта	19
Создание виртуального окружения.....	21
Написание скрипта запуска приложения на новой системе	24
Вывод.....	25

ПРАКТИЧЕСКАЯ РАБОТА №2. ОСНОВЫ РАБОТЫ С BASH SCRIPT АМИ

Введение

Цель работы: получить практические навыки по написанию Bash-скриптов и разворачиванию проектов с их помощью.

В рамках работы рассматриваются основы создания Bash-скриптов, включая работу с переменными, управляющими конструкциями, циклами и операциями с файлами. Также изучается процесс подготовки системы для запуска приложения: определение зависимостей проекта, создание виртуального окружения и автоматизация настройки необходимого программного обеспечения.

Часть 1. Базовые Bash скрипты

Bash-скрипты представляют из себя ни что иное как последовательность команд командной строки, объединенных в один файл для решения какой-либо задачи.

Любой скрипт начинается с так называемой шебанг строки, которая указывает на расположение исполняемого файла той оболочки, для которой мы хотим написать скрипт. В нашем случае это bash, следовательно данная строка будет выглядеть следующим образом:

```
#!/bin/bash
```

После чего уже следует непосредственно сам скрипт.

В первый скрипт запишем последовательность:

```
#!/bin/bash  
# This is a comment  
pwd
```

Результат выполнения скрипта представлен на рис. 1.

```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ chmod +x ./myscript

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript
/c/Users/borga/Desktop/Студ/ТРПП
```

Рисунок 1 – Запуск первого скрипта

Переменные

Для обращения к переменным среды следует использовать следующую конструкцию: ***\$ИМЯ_ПЕРЕМЕННОЙ***.

Вывод скрипта с командами:

```
#!/bin/bash
#display user home
echo "Home for the current user is: $HOME"
```

представлен на рис. 2.

```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript
Home for the current user is: /c/Users/borga
```

Рисунок 2 – Выполнение скрипта с переменной

Результат работы скрипта с командами:

```
#!/bin/bash
# testing variables
grade=5
person="Adam"
echo "$person is a good boy, he is in grade $grade"
```

представлен на рис. 3.

```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript
Adam is a good boy, he is in grade 5
```

Рисунок 3 – Выполнение скрипта с несколькими переменными

Подстановка команд

Одна из самых полезных возможностей bash-скриптов — это возможность извлекать информацию из вывода команд и назначать её переменной, что позволяет использовать эту информацию где угодно в файле

сценария.

Сделать это можно двумя способами:

- с помощью значка обратного апострофа «`»;
- с помощью конструкции `$()`.

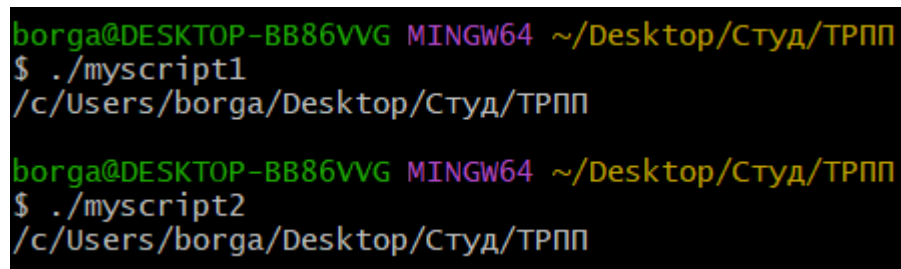
Скрипт на основе первого варианта:

```
#!/bin/bash
mydir=`pwd`
echo $mydir
```

Скрипт на основе второго варианта:

```
#!/bin/bash
mydir=$(pwd)
echo $mydir
```

Выполнение обоих вариантов скрипта представлено на рис. 4.



```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscrip1
/c/Users/borga/Desktop/Студ/ТРПП

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscrip2
/c/Users/borga/Desktop/Студ/ТРПП
```

Рисунок 4 – Выполнение разных вариантов скрипта с подстановкой команд

Как видно из рисунка, результат выполнения идентичный в любом из случаев.

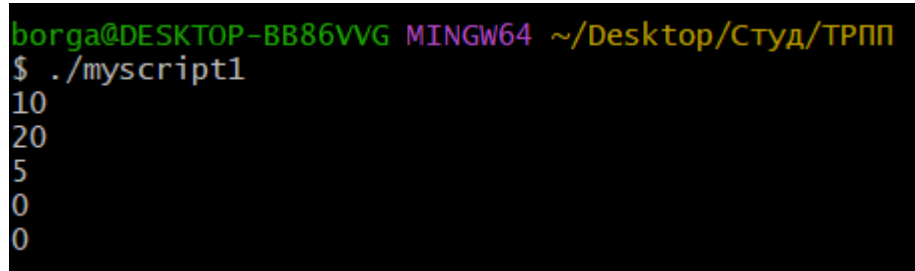
Математические операции

Для выполнения математических операций в файле скрипта можно использовать конструкцию вида `$(( a + b ))`:

```
#!/bin/bash
var1=$(( 5 + 5 ))
echo $var1
var2=$(( $var1 * 2 ))
echo $var2
var3=$(( $var2 / 4 ))
echo $var3
var4=$(( $var3 % 5 ))
echo $var4
var5=$(( $var4 ** 2 ))
```

```
echo $var5
```

Выполнение скрипта представлено на рис. 5.



```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript1
10
20
5
0
0
```

Рисунок 5 – Выполнение скрипта с математическими операциями

Управляющая конструкция if-then-else

Для управления потоком исполнения команд в bash-скриптах можно использовать управляющую конструкцию if-then-else. Работает она как во всех языках программирования, однако имеет свои особенности. Вот её подробный синтаксис:

```
if команда1
then
команды
elif команда2
then
команды
else
команды
fi
```

Пример скрипта с if-then-else:

```
#!/bin/bash
user=anotherUser
if grep $user /etc/passwd
then
echo "The user $user Exists"
else
echo "The user $user doesn't exist"
fi
```

Пример скрипта с elif:

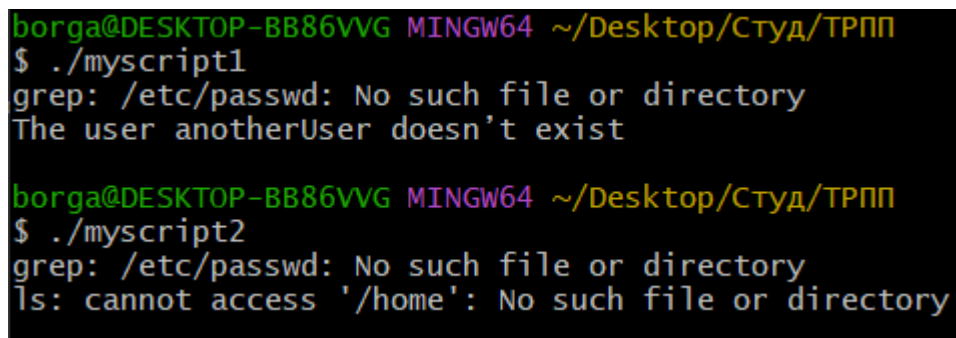
```
#!/bin/bash
user=anotherUser
if grep $user /etc/passwd
then
```

```

echo "The user $user Exists"
elif ls /home
then echo "The user doesn't exist but anyway there
is a
directory under /home"
fi

```

Процесс выполнения скриптов представлен на рис. 6.



```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript1
grep: /etc/passwd: No such file or directory
The user anotherUser doesn't exist

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript2
grep: /etc/passwd: No such file or directory
ls: cannot access '/home': No such file or directory

```

Рисунок 6 – Выполнение скриптов с условным ветвлением

Сравнение чисел

В отличие от привычных операций сравнения в языках программирования bash-скрипты обладают специфическим синтаксисом операций сравнения. Например, для чисел применяются следующие операции:

- ***n1 -eq n2*** – Возвращает истинное значение, если n1 равно n2.
- ***n1 -ge n2*** – Возвращает истинное значение, если n1 больше или равно n2.
- ***n1 -gt n2*** – Возвращает истинное значение, если n1 больше n2.
- ***n1 -le n2*** – Возвращает истинное значение, если n1 меньше или равно n2.
- ***n1 -lt n2*** – Возвращает истинное значение, если n1 меньше n2.
- ***n1 -ne n2*** – Возвращает истинное значение, если n1 не равно n2.

При использовании подобных операций выражения необходимо заключать в квадратные скобки, например:

```

#!/bin/bash
val1=6

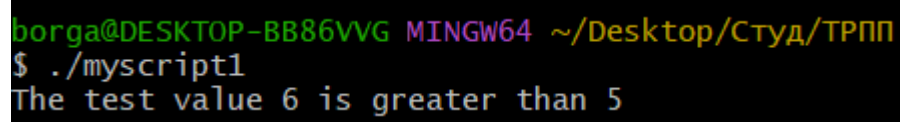
```

```

if [ $val1 -gt 5 ]
then
echo "The test value $val1 is greater than 5"
else
echo "The test value $val1 is not greater than 5"
fi

```

Выполнение скрипта для значения 6 представлено на рис. 7.



```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript1
The test value 6 is greater than 5

```

Рисунок 7 – Выполнение скрипта со сравнением

Сравнение строк

Для сравнения строк используются следующие операторы:

- ***str1 = str2*** – Проверяет строки на равенство, возвращает истину, если строки идентичны.
- ***str1 != str2*** – Возвращает истину, если строки не идентичны.
- ***str1 < str2*** – Возвращает истину, если str1 меньше, чем str2.
- ***str1 > str2*** – Возвращает истину, если str1 больше, чем str2.
- ***-n str1*** – Возвращает истину, если длина str1 больше нуля.
- ***-z str1*** – Возвращает истину, если длина str1 равна нулю.

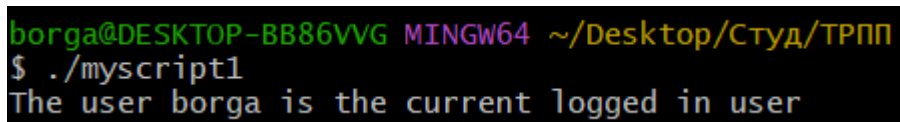
Пример сравнения строк:

```

#!/bin/bash
user ="borga"
if [ $user = $USER ]
then
echo "The user $user is the current logged in user"
fi

```

Применение скрипта представлено на рис. 8.



```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript1
The user borga is the current logged in user

```

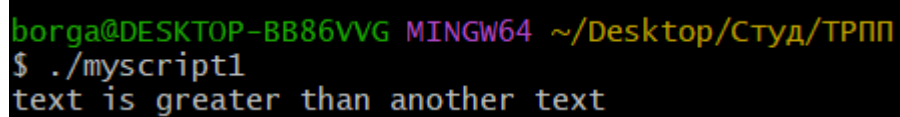
Рисунок 8 – Выполнение скрипта со сравнением строк

При работе с операторами < и > есть несколько особенностей, которые стоит учитывать. Во-первых, их необходимо экранировать при помощи

символа \. Во-вторых, необходимо заключать имена переменных в двойные кавычки "\$val2". Пример работы с подобным оператором:

```
#!/bin/bash
val1=text
val2="another text"
if [ "$val1" \> "$val2" ]
then
echo "$val1 is greater than $val2"
else
echo "$val1 is less than $val2"
fi
```

Выполнение скрипта представлено на рис. 9.



```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript1
text is greater than another text
```

Рисунок 9 – Пример выполнения скрипта с использованием оператора > для строк

Проверки файлов

Наиболее используемыми командами для bash скриптов являются команды проверки файлов.

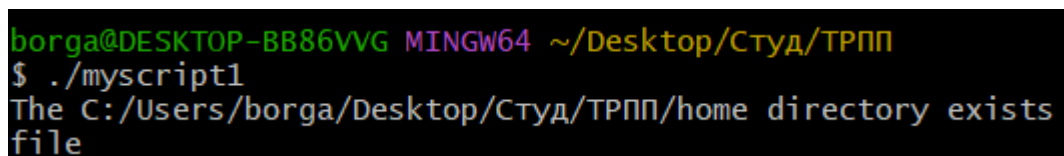
- **-d file** – Проверяет, существует ли файл, и является ли он директорией;
- **-e file** – Проверяет, существует ли файл;
- **-f file** – Проверяет, существует ли файл, и является ли он файлом;
- **-r file** – Проверяет, существует ли файл, и доступен ли он для чтения;
- **-s file** – Проверяет, существует ли файл, и не является ли он пустым;
- **-w file** – Проверяет, существует ли файл, и доступен ли он для записи;
- **-x file** – Проверяет, существует ли файл, и является ли он исполняемым;

- *file1 -nt file2* – Проверяет, новее ли file1, чем file2;
- *file1 -ot file2* – Проверяет, старше ли file1, чем file2;
- *-O file* – Проверяет, существует ли файл, и является ли его владельцем текущий пользователь;
- *-G file* – Проверяет, существует ли файл, и соответствует ли его идентификатор группы идентификатору группы текущего пользователя.

Пример скрипта, в котором используются эти команды.

```
#!/bin/bash
mydir=C:/Users/borga/Desktop/Студ/ТРПП/home
if [ -d $mydir ]
then
echo "The $mydir directory exists"
cd $mydir
ls
else
echo "The $mydir directory does not exist"
fi
```

Пример запуска скрипта представлен на рис. 10.



```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript1
The C:/Users/borga/Desktop/Студ/ТРПП/home directory exists
file
```

Рисунок 10 – Запуск скрипта с проверкой существования директории

Операторы цикла

Bash-скрипты поддерживают несколько вариантов циклов для перебора последовательностей значений. В число этих циклов входят цикл *for* и цикл *while*.

Для начала рассмотрим цикл *for*. Базовая структура такого цикла выглядит следующим образом.

```
for var in list
do
команды
done
```

Самый простой пример – перебор списка простых значений:

```
#!/bin/bash
for var in $first second third fourth fifth
do
echo The $var item
done
```

Пример выполнения скрипта представлен на рис. 11.

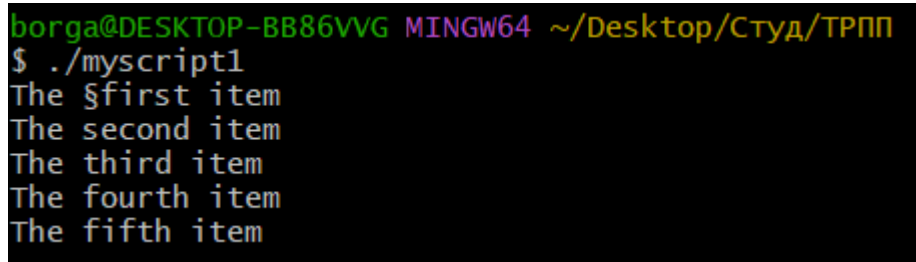


Рисунок 11 – Скрипт с циклом for

Для перебора сложных значений необходимо заключать эти значения в строковые литералы – двойные кавычки, к примеру:

```
#!/bin/bash
for var in first "the second" "the third" "I'll do
it"
do
echo "This is: $var"
done
```

Пример выполнения скрипта представлен на рис. 12.

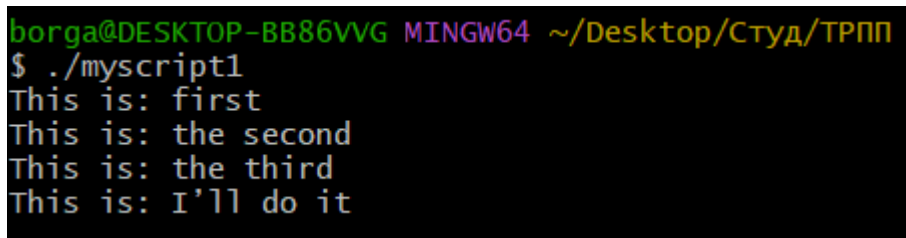


Рисунок 12 – Скрипт с перебором сложных значений

Список для цикла for может быть получен из результата выполнения какой-либо команды. Как пример – перебор вывода команды cat:

```
#!/bin/bash
file="myfile"
for var in $(cat $file)
do
echo " $var"
done
```

Пример выполнения скрипта представлен на рис. 13.

```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript1
a
b
c
d
e
f
```

Рисунок 13 – Скрипт с чтением файла по словам

Однако проблема такого вывода в том, что файл будет обрабатываться по словам, а не по строкам.

Это связано с тем, что оболочка `bash` считает разделителем строки следующий набор символов:

- пробел;
- знак табуляции;
- знак перевода строки.

Для того, чтобы задать разделитель необходимо использовать переменную окружения `IFS` (Internal Field Separator). Пример того, как можно выполнить итерацию лишь по строкам файла:

```
#!/bin/bash
file="myfile"
IFS=$'\n'
for var in $(cat $file)
do
echo " $var"
done
```

Пример выполнения скрипта представлен на рис. 14.

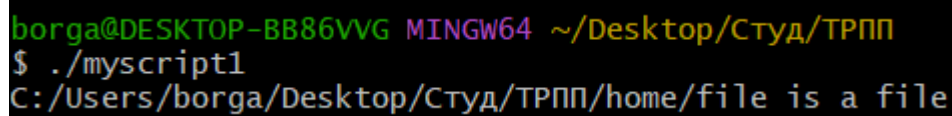
```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript1
a b c d e f
g h i j k l
m n o p q r
s t u v w x
y z
```

Рисунок 14 – Скрипт с чтением файла по строкам

Для обхода файлов, находящихся в директориях, также используется цикл `for`. Вот пример такого обхода с выводом списка файлов и директорий:

```
#!/bin/bash
for file in C:/Users/borga/Desktop/Студ/ТРПП/home/*
do
if [ -d "$file" ]
then
echo "$file is a directory"
elif [ -f "$file" ]
then
echo "$file is a file"
fi
done
```

Пример выполнения скрипта представлен на рис. 15.



```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript1
C:/Users/borga/Desktop/Студ/ТРПП/home/file is a file
```

Рисунок 15 – Скрипт с обходом файлов в директории

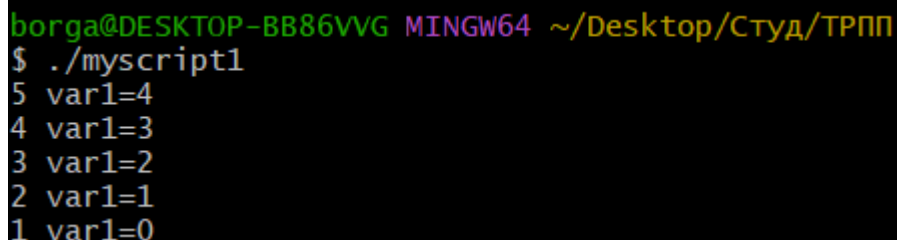
Вторая разновидность циклов – цикл while:

```
while команда проверки условия
do
другие команды
done
```

Пример скрипта с таким циклом:

```
#!/bin/bash
var1=5
while [ $var1 -gt 0 ]
do
echo $var1 var1=$(( $var1 - 1 ))
var1=$(( $var1 - 1 ))
done
```

Пример выполнения скрипта представлен на рис. 16.



```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript1
5 var1=4
4 var1=3
3 var1=2
2 var1=1
1 var1=0
```

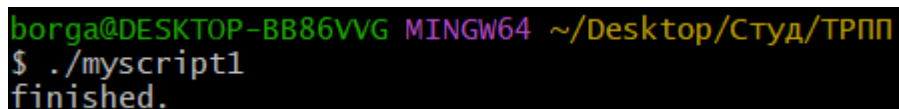
Рисунок 16 – Скрипт с циклом while

Для управления циклами также могут применяться привычные команды операторы *break* и *continue*.

Результат работы цикла также могут быть выведены в файл при помощи перенаправления вывода. Вот пример такого скрипта.

```
#!/bin/bash
for (( a = 1; a < 10; a++ ))
do
echo "Number is $a"
done > myfile.txt
echo "finished."
```

Пример выполнения скрипта представлен на рис. 17.



```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript1
finished.
```

Рисунок 17 – Скрипт с выводом в файл

Выполнение заданий

1. Сценарий, который выводит дату, время, список зарегистрировавшихся пользователей, и uptime системы и сохраняет эту информацию в файл:

```
#!/bin/bash
output_file="system_info.txt"
date=$(date)
users=$(powershell.exe -Command "Get-Process |
Select-Object -ExpandProperty SessionId -Unique"
2>&1)
uptime=$(powershell.exe -Command "Get-WmiObject
Win32_OperatingSystem | Select-Object -
ExpandProperty LastBootUpTime")
echo "System Information:" > $output_file
echo "Date and Time: $date" >> $output_file
echo "Logged in users:" >> $output_file
echo "$users" >> $output_file
wmic useraccount get name >> $output_file
echo "System Uptime: $uptime" >> $output_file
echo "Information saved to $output_file"
```

Пример выполнения скрипта представлен на рис. 18.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП
$ . myscript3
Information saved to system_info.txt

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП
$ cat system_info.txt
System Information:
Date and Time: Sat Mar 22 09:42:44 RTZ 2025
Logged in users:
20
0
Name
borga
DefaultAccount
WDAGUtilityAccount
4<8=8AB@0B>@
>ABL
System Uptime: 20250315004141.180893+180

```

Рисунок 18 – Выполнение скрипта с информацией о системе

2. Сценарий, который выводит содержимое любого каталога или сообщение о том, что его не существует:

```

#!/bin/bash
if [ -z "$1" ]; then
    echo "Usage: $0 <directory_path>"
    exit 1
fi
if [ -d "$1" ]; then
    echo "Contents of directory $1:"
    ls -l "$1"
else
    echo "Directory $1 does not exist."
fi

```

Пример выполнения скрипта представлен на рис. 19.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП
$ ./myscript2 home
Contents of directory home:
total 0
-rw-r--r-- 1 borga 197609 0 Feb 22 18:06 file

```

Рисунок 19 – Пример выполнения скрипта, выводящего содержимое директории

3. Сценарий, который с помощью цикла прочитает файл и выведет его содержимое:

```

#!/bin/bash

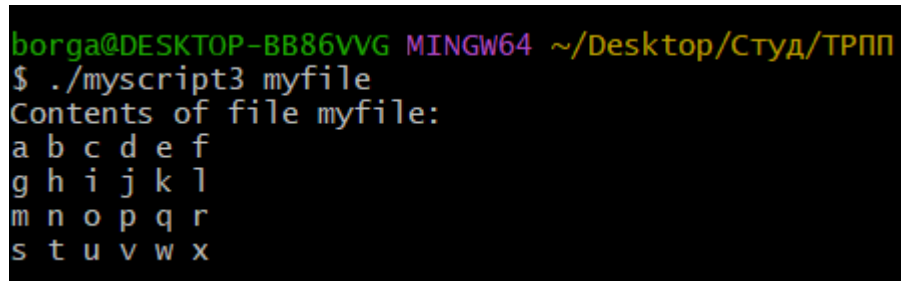
```

```

if [ -z "$1" ]; then
    echo "Usage: $0 <file_path>"
    exit 1
fi
if [ -f "$1" ]; then
    echo "Contents of file $1:"
    while IFS= read -r line; do
        echo "$line"
    done < "$1"
else
    echo "File $1 does not exist."
fi

```

Пример выполнения скрипта представлен на рис. 20.



```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript3 myfile
Contents of file myfile:
a b c d e f
g h i j k l
m n o p q r
s t u v w x

```

Рисунок 20 – Выполнения скрипта, выводящего содержимое файла

4. Сценарий, который с помощью цикла выведет список файлов и директорий из текущего каталога, укажет, что есть файл, а что директория:

```

#!/bin/bash
for item in *; do
    if [ -d "$item" ]; then
        echo "$item is a directory"
    elif [ -f "$item" ]; then
        echo "$item is a file"
    fi
done

```

Пример выполнения скрипта представлен на рис. 21.


```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript4
1ПР_Боргачев_ТМ.docx is a file
1ПР_Боргачев_ТМ.pdf is a file
2ПР_Боргачев_ТМ.docx is a file
Access_Control is a directory
H is a directory
New is a directory
New_one is a directory
SADT is a directory
Test is a directory
Variant is a directory
home is a directory
myfile is a file
myfile.txt is a file
myscript1 is a file
myscript2 is a file
myscript3 is a file
myscript4 is a file
myscript5 is a file
myscript6 is a file
python-cheatsheet is a directory
repository is a directory
system_info.txt is a file
~$P_Боргачев_ТМ.docx is a file
~$WRL0005.tmp is a file

```

Рисунок 21 – Выполнение скрипта, разделяющий файлы и директории

5. Сценарий, который подсчитает объем диска, занимаемого директорией. В качестве директории можно выбрать любую директорию в системе:

```

#!/bin/bash
if [ -z "$1" ]; then
    echo "Usage: $0 <directory_path>"
    exit 1
fi
if [ -d "$1" ]; then
    size=$(du -sh "$1" | cut -f1)
    echo "The size of directory $1 is: $size"
else
    echo "Directory $1 does not exist."
fi

```

Пример выполнения скрипта представлен на рис. 22.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript5 ~/Desktop
The size of directory /c/Users/borga/Desktop is: 7.7G

```

Рисунок 22 – Выполнение скрипта, подсчитывающего размер директории

6. Сценарий, который выведет список всех исполняемых файлов в директории, для которых у текущего пользователя есть права на исполнение:

```

#!/bin/bash
if [ -z "$1" ]; then
    echo "Usage: $0 <directory_path>"
    exit 1
fi
if [ -d "$1" ]; then
    echo "Executable files in directory $1:"
    for file in "$1"/*; do
        if [ -x "$file" ]; then
            echo "$(basename "$file")"
        fi
    done
else
    echo "Directory $1 does not exist."
fi

```

Пример выполнения скрипта представлен на рис. 23.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ ./myscript6 ~/Desktop
Executable files in directory /c/Users/borga/Desktop:
Command Prompt.lnk
DS4Windows.lnk
DeepL.lnk
Discord.lnk
MTS Link.lnk
MusicBee.lnk
PhotoshopPortable (2).lnk
RStudio.lnk
Spotify.lnk
Translumo.exe – ярлык.lnk
Visual Studio Code.lnk
control.exe
uTorrent Web.lnk
uTorrent.lnk
zapret.lnk
Браузер Opera GX.lnk
Студ
Яндекс Музыка.lnk
папка

```

Рисунок 23 – Выполнение скрипта, выводящего все разрешенные исполняемые файлы

Таким образом, все задания были выполнены с помощью Bash скриптов.

Часть 2. Развертка и запуск проекта при помощи Bash Script

Определение зависимостей проекта

После загрузки проекта, его необходимо распаковать командой *tar -xvf имя_архива имя_директории_для_распаковки*. Пример распаковки представлен на рис. 24.

```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ tar -xvf blocknote-master.tar.gz blocknote-master
blocknote-master/
blocknote-master/._.DS_Store
blocknote-master/.DS_Store
blocknote-master/appengine/
blocknote-master/._.gitignore
blocknote-master/.gitignore
blocknote-master/static/
blocknote-master/._.gitattributes
blocknote-master/.gitattributes
blocknote-master/templates/
blocknote-master/._manage.py
blocknote-master/manage.py
blocknote-master/apps/
blocknote-master/apps/._.DS_Store
blocknote-master/apps/.DS_Store
blocknote-master/apps/todoapp/
blocknote-master/apps/main/
blocknote-master/apps/main/migrations/
blocknote-master/apps/main/._services.py
blocknote-master/apps/main/services.py
blocknote-master/apps/main/._models.py
blocknote-master/apps/main/models.py
blocknote-master/apps/main/.__init__.py
blocknote-master/apps/main/__init__.py
blocknote-master/apps/main/._apps.py
blocknote-master/apps/main/apps.py
blocknote-master/apps/main/._forms.py
blocknote-master/apps/main/forms.py
blocknote-master/apps/main/._admin.py
blocknote-master/apps/main/admin.py
blocknote-master/apps/main/._tests.py
blocknote-master/apps/main/tests.py
blocknote-master/apps/main/._urls.py
blocknote-master/apps/main/urls.py
blocknote-master/apps/main/._views.py
blocknote-master/apps/main/views.py
```

Рисунок 24 – Распаковка проекта

Если открыть файл `manage.py`, можно увидеть зависимость проекта от модулей `django`, `pillow`, `base` и `dotenv`, а так же сам проект зависит от модулей `ckeditor`, `simple-history` и `tinymce` – добавим их в `requirements.txt`.

Таким образом `requirements.txt` будет выглядеть как представлено на рис. 25.

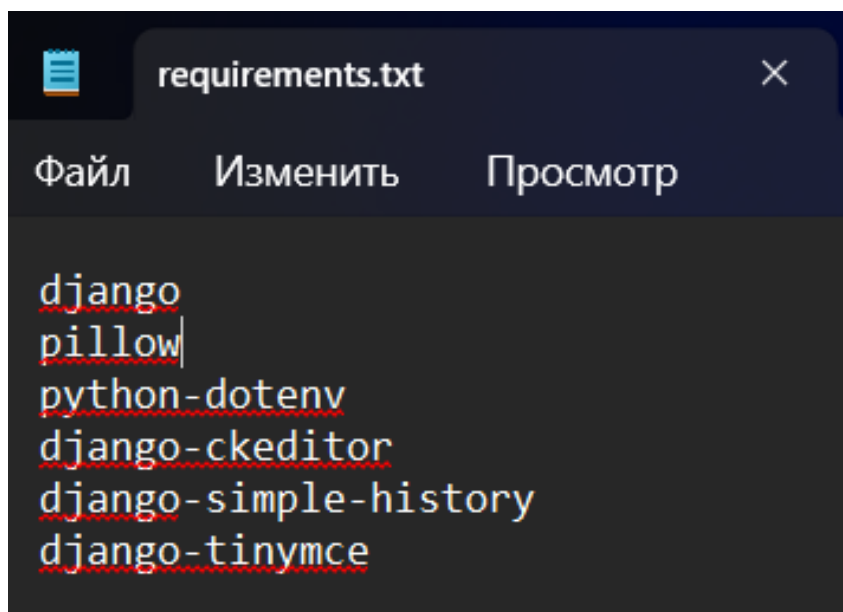


Рисунок 25 – Список зависимостей

Создание виртуального окружения

Создание виртуального окружения с помощью языка Python представлено на рис. 26.

```
borga@DESKTOP-BB86VVG MINGW64 ~  
$ cd Desktop/Студ/ТРПП  
  
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП  
$ python -m venv ENV  
  
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП  
$ source ENV/Scripts/activate  
(ENV)  
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП  
$
```

Рисунок 26 – Виртуальное окружение python

Пример установки зависимостей из requirements.txt представлено на рис. 27.

```
(venv)
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ pip install -r blocknote-master/requirements.txt
Requirement already satisfied: django in c:\users\bo
Requirement already satisfied: pillow in c:\users\bo
Requirement already satisfied: python-dotenv in c:\u
Requirement already satisfied: django-ckeditor in c:
Requirement already satisfied: django-simple-history
Requirement already satisfied: django-tinymce in c:\
Requirement already satisfied: asgiref<4,>=3.8.1 in
Requirement already satisfied: sqlparse>=0.3.1 in c:
Requirement already satisfied: tzdata in c:\users\bo
Requirement already satisfied: django-js-asset>=2.0
```

Рисунок 27 – Установка зависимых модулей

Добавим все команды в скрипт для настройки виртуального окружения.

Конечный скрипт будет выглядеть так:

```
#!/bin/bash

if ! command -v python &> /dev/null; then
    echo "Python не установлен. Установите Python и
добавьте его в PATH."
    exit 1
fi

if [ ! -f blocknote-master.tar.gz ]; then
    echo "Скачивание проекта..."
    wget
https://www.dropbox.com/s/ija7ax3sj6ysb0p/blocknote-
master.tar.gz
else
    echo "Архив проекта уже скачан."
fi

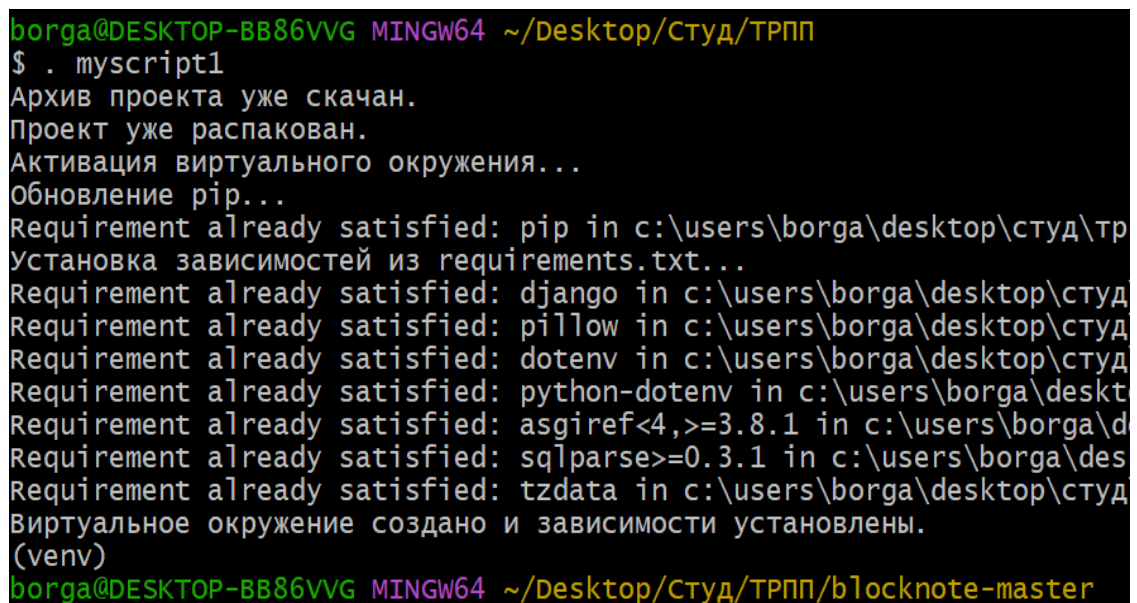
if [ ! -d blocknote-master ]; then
    echo "Распаковка архива..."
    tar -xvf blocknote-master.tar.gz
else
    echo "Проект уже распакован."
fi
```

```

cd blocknote-master
if [ ! -d venv ]; then
    python -m venv venv
fi
echo "Активация виртуального окружения..."
source venv/Scripts/activate
echo "Обновление pip..."
python -m pip install --upgrade pip
echo "Установка зависимостей из
requirements.txt..."
if [ -f requirements.txt ]; then
    pip install -r requirements.txt
else
    echo "Файл requirements.txt не найден."
    exit 1
fi
echo "Виртуальное окружение создано и зависимости
установлены."

```

Пример запуска скрипта представлен на рис. 28.



```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ . myscript1
Архив проекта уже скачан.
Проект уже распакован.
Активация виртуального окружения...
Обновление pip...
Requirement already satisfied: pip in c:\users\borga\desktop\студ\тр
Установка зависимостей из requirements.txt...
Requirement already satisfied: django in c:\users\borga\desktop\студ
Requirement already satisfied: pillow in c:\users\borga\desktop\студ
Requirement already satisfied: dotenv in c:\users\borga\desktop\студ
Requirement already satisfied: python-dotenv in c:\users\borga\deskt
Requirement already satisfied: asgiref<4,>=3.8.1 in c:\users\borga\d
Requirement already satisfied: sqlparse>=0.3.1 in c:\users\borga\des
Requirement already satisfied: tzdata in c:\users\borga\desktop\студ
Виртуальное окружение создано и зависимости установлены.
(venv)
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/blocknote-master

```

Рисунок 28 – Запуск скрипта для создания виртуального окружения

Написание скрипта запуска приложения на новой системе

Скрипт для запуска приложения выглядит так:

```
#!/bin/bash
python manage.py makemigrations
python manage.py migrate
python manage.py runserver
```

Пример запуска скрипта представлен на рис. 29.

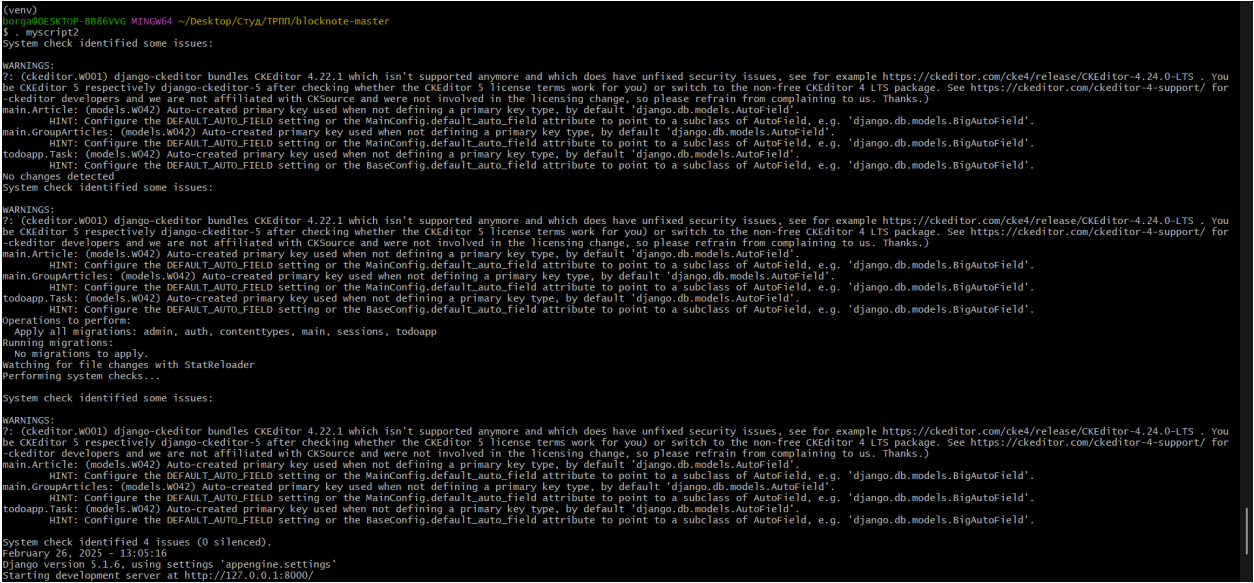


Рисунок 29 – Запуск приложения

Результат запуска приложения представлен на рис. 30.

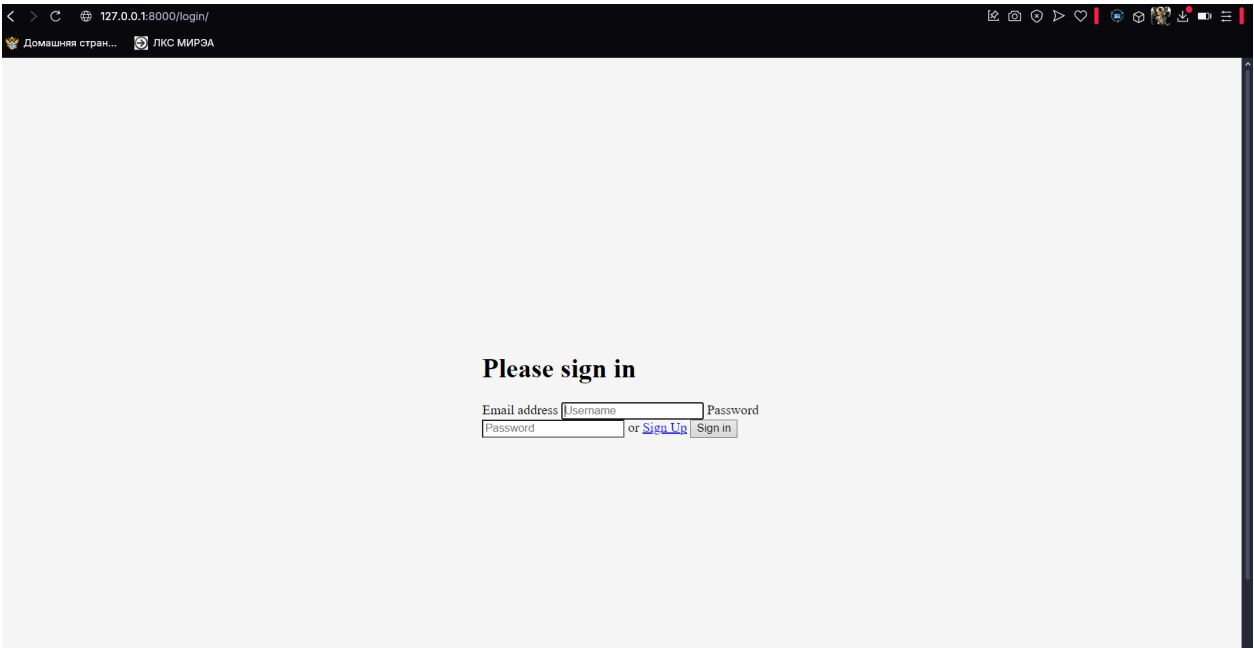


Рисунок 30 – Результат запуска приложения

Таким образом, скрипт отработал корректно, запустив приложение.

Вывод

В рамках практической работы были изучены основы написания Bash-скриптов и их применение для развертывания проектов. Были рассмотрены базовые команды и конструкции bash-скриптов, включая работу с переменными, условными операторами, циклами и операциями с файлами.

Полученные навыки были применены для создания скрипта автоматической настройки проекта, который:

- проверяет наличие необходимых зависимостей;
- скачивает и распаковывает проект;
- создает виртуальное окружение Python;
- устанавливает зависимости проекта;
- выполняет миграции и запускает приложение.

Также были выполнены практические задания по созданию различных bash-скриптов для работы с файловой системой и получения информации о системе.

Работа позволила получить практические навыки по автоматизации процессов развертывания проектов с помощью bash-скриптов, что является важным аспектом современной разработки программного обеспечения.